YOUR NAME

Final Project
ECE 125, Due:12/10/2025 (11:59 pm)

Problem: Deep Learning Identification of a 4th-Order Linear System

In this project, you will learn the dynamics of a stable fourth-order linear system using a deep neural network (DNN). You are given the continuous-time state-space model

$$\begin{aligned}
\dot{x}_1(t) &= x_2(t),\\
\dot{x}_2(t) &= x_3(t),\\
\dot{x}_3(t) &= x_4(t),\\
\dot{x}_4(t) &= -24\,x_1(t) - 50\,x_2(t) - 35\,x_3(t) - 10\,x_4(t) + u(t),\\
y(t) &= x_1(t),
\end{aligned}$$

where $x(t) \in \mathbb{R}^4$ is the state vector, $u(t) \in \mathbb{R}$ is the input, and $y(t) \in \mathbb{R}$ is the measured output. The characteristic polynomial of the system is $(s+1)(s+2)(s+3)(s+4)$, so the system is asymptotically stable.

(a) Simulation and data generation

- Choose a sampling time $T_s \leq 0.005$ s (at most 5 milliseconds). Simulate the system for a total time T_{final} long enough to collect at least $N \geq 1000$ samples of the form

$$\{x_k, u_k, y_k\}_{k=0}^{N-1}, \quad \text{with } x_k = x(kT_s),\ u_k = u(kT_s),\ y_k = y(kT_s).$$

- Use a sufficiently rich input $u(t)$, for example a pseudo-random binary sequence (PRBS), a sum of sinusoids, or a sequence of step changes. Clearly describe your choice of $u(t)$ and initial state $x(0)$ in your report.
- Store the data as a dataset where the input features at time k are

$$z_k = \begin{bmatrix} x_k \\ u_k \end{bmatrix} \in \mathbb{R}^5,$$

  and the supervised learning target is y_k.

(b) Train/test split and preprocessing

- Use 80% of the data for training and 20% for testing. For example, if N samples are collected, you may take the first $N_{\text{train}} = \lfloor 0.8N \rfloor$ samples for training and the remaining $N_{\text{test}} = N - N_{\text{train}}$ samples for testing (or randomly shuffle the samples before splitting).
- Normalize or standardize the input features z_k and output y_k (for example, subtract the mean and divide by the standard deviation, or scale to $[-1, 1]$). Clearly state the normalization method and apply the same transformation to both training and test sets.

(c) Baseline deep neural network design

Design a feedforward deep neural network (DNN) to approximate the static mapping from (x_k, u_k) to y_k:

$$\hat{y}_k = f_\theta(z_k),$$

where f_θ is parameterized by weights and biases θ.

- Use at least 4 hidden layers and at least 16 neurons in each hidden layer (for example, 5–16–16–16–16–1).
- Choose an activation function (e.g., ReLU) for the hidden layers, and a linear activation for the output layer.
- Train the network using the training set (z_k, y_k) to minimize the mean squared error (MSE)

$$\text{MSE} = \frac{1}{N_{\text{train}}} \sum_{k=1}^{N_{\text{train}}} (y_k - \hat{y}_k)^2.$$

- After training, evaluate the test MSE on the test set and compute any additional performance metric you find useful (e.g., root mean squared error, normalized MSE, or coefficient of determination R^2).
- Plot on a single figure the MSE as a function of Epochs and on another figure, the true output y_k and the predicted output $\hat{y}_k$ for the test samples versus time index k, and briefly comment on the quality of the fit.

(d) Impact of the amount of training data
Investigate how the amount of training data affects the accuracy of the neural network.

- Using the same total dataset and test set as in part (c), create three training scenarios:
  1. *Case 1 (100% training data):* Use all N_{train} training samples.
  2. *Case 2 (80% of training data):* Randomly keep only $0.8N_{\text{train}}$ training samples and discard the rest.
  3. *Case 3 (60% of training data):* Randomly keep only $0.6N_{\text{train}}$ training samples.
- For each case, retrain the same DNN architecture (same number of layers, neurons, and activation function) from scratch.
- Report the training MSE and test MSE for each case in a table. Discuss how reducing the training data from 100% to 80% to 60% affects overfitting, underfitting, and generalization.

(e) Impact of activation functions
Investigate the effect of different activation functions on performance.

- Fix the training data to Case 1 (use all N_{train} training samples) and keep the same network depth and width as in part (c).
- Train three separate networks using different activation functions in the hidden layers:
  1. ReLU,
  2. Hyperbolic tangent (tanh),
  3. Sigmoid (or another activation of your choice, e.g., leaky ReLU).
- For each activation function, report training and test MSE and show plots of y_k versus $\hat{y}_k$ on the test set.
- Compare and discuss which activation function performs better and why (e.g., gradient saturation, nonlinearity type, training speed).

(f) Impact of number of neurons per layer
Investigate the effect of the model size (number of neurons per layer) on prediction accuracy.

- Fix the activation function (e.g., ReLU) and training data (Case 1: all N_{train} samples).
- Compare at least three architectures with the same number of hidden layers (at least 4), but different widths, for example:

$$5\text{–}8\text{–}8\text{–}8\text{–}8\text{–}1, \quad 5\text{–}16\text{–}16\text{–}16\text{–}16\text{–}1, \quad 5\text{–}32\text{–}32\text{–}32\text{–}32\text{–}1.$$

- For each architecture, train the network and report training and test MSE.
- Discuss the trade-off between model complexity and performance. Comment on signs of underfitting (too few neurons) and overfitting (too many neurons relative to data).

(g) Minimum number of training samples for 5% error
Finally, investigate how few training samples are needed for the network to achieve a given accuracy level.

- Starting from your best-performing architecture and activation function from parts (c)–(f), progressively reduce the size of the training set by using a shorter simulation time or by sub-sampling the data. Keep the test set fixed for fair comparison.

- For each training set size $N_{\text{train}}^{(i)}$ (e.g., $N_{\text{train}}^{(i)} \in \{800, 600, 400, 200, 100, \dots\}$), train the network and compute the test mean squared error

$$\text{MSE}_{\text{test}}^{(i)} = \frac{1}{N_{\text{test}}} \sum_{k=1}^{N_{\text{test}}} (y_k - \hat{y}_k)^2.$$

- Define "acceptable accuracy" as predicting the outputs with less than 5% error, interpreted as a test MSE below 5% (for example, if y is normalized, require $\text{MSE}_{\text{test}}^{(i)} \leq 0.05$).
- Determine the *minimum* number of training samples $N_{\text{train,min}}$ for which the network satisfies this condition. Report:
  1. the value of $N_{\text{train,min}}$,
  2. the corresponding test MSE,
  3. a plot comparing y_k and $\hat{y}_k$ on the test set for this minimal training size.
- Briefly discuss how the performance degrades as the number of training samples decreases and what this implies about data requirements for learning dynamical systems.

Deliverables: Your report should include (i) a brief description of the problem (with typed equations only) and simulation setup, (ii) network architectures and hyperparameters used and show the structure of your neural network (draw using Microsoft PowerPoint, visio, or other drawing software, see online examples to learn) and clearly show the input layer, hidden layer with number of neurons, output layer, and weights, write the necessary formulas for forward and backward propagation, (iii) tables summarizing MSE results for parts (d)–(f), (iv) plots of y_k and $\hat{y}_k$ on the test set, and (v) a concise discussion answering the question: *What is the minimum number of training samples your neural network needs to correctly predict the outputs with less than 5% error?*

Please make sure to include a copy of your code at the end of your report